Il Processore Mic-1

Implementazione

Prof. Nicola Mazzocca
Ing. Alberto Moriconi

Corso di Architettura dei Sistemi di Elaborazione
Università degli Studi di Napoli Federico II

Macchine a Stack

Il processore Mic-1 è una macchina *a stack*, e non dispone di registri generali.

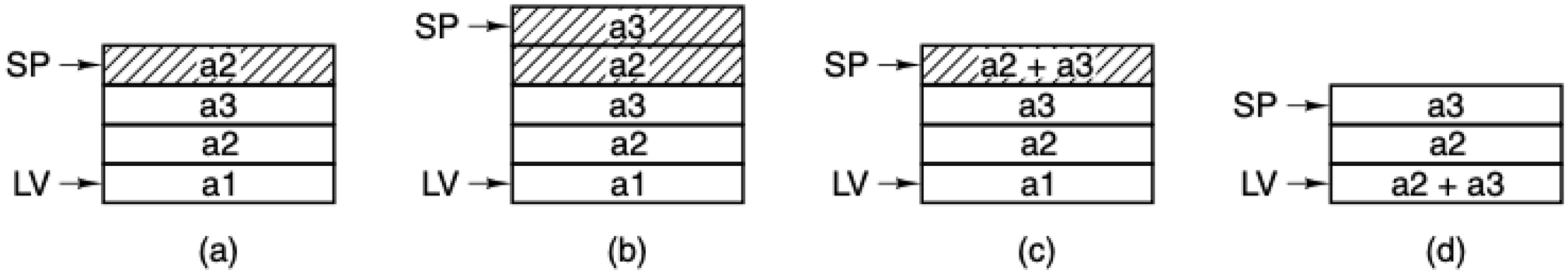
Consideriamo l'istruzione Java:

```
a1 = a2 + a3;
```

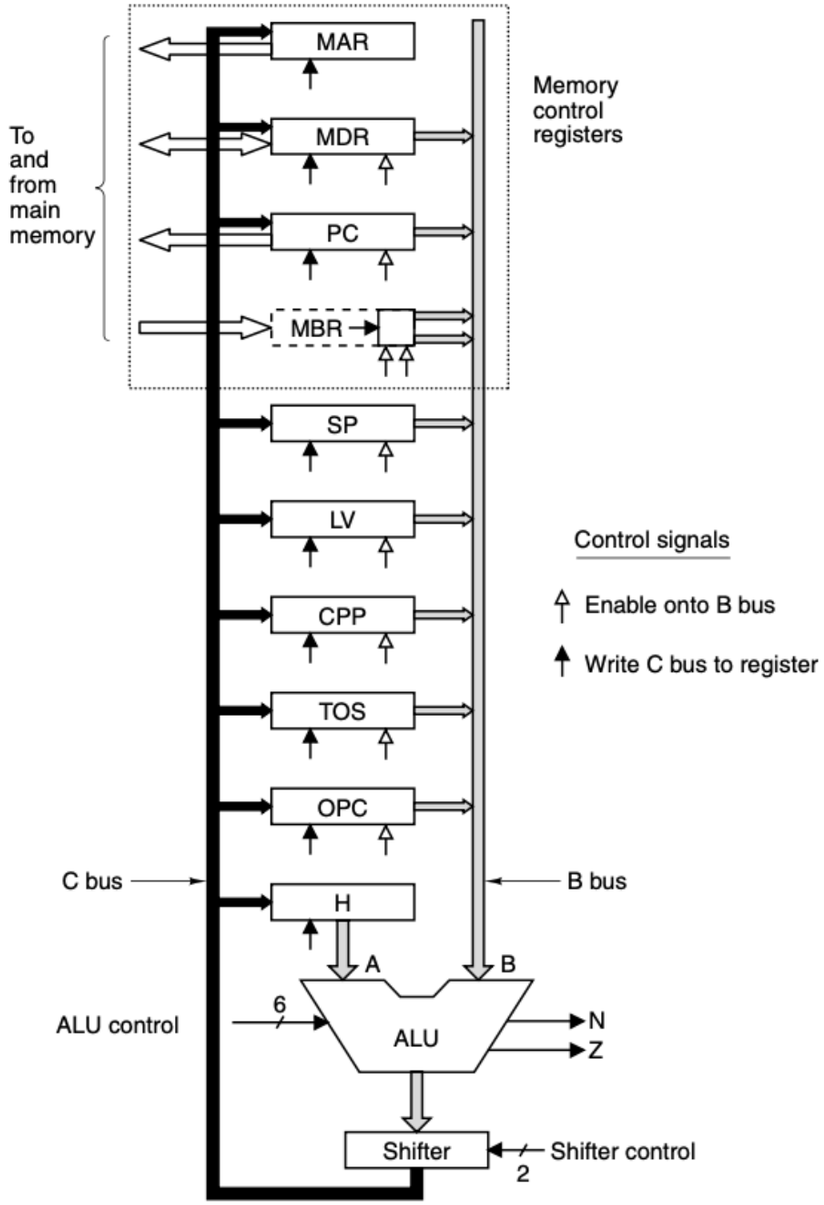
In assembly IJVM, è tradotta come:

```
ILOAD a2      (a)
ILOAD a3      (b)
IADD          (c)
ISTORE a1     (d)
```

Osserviamo come evolve lo stack:



Unità Operativa

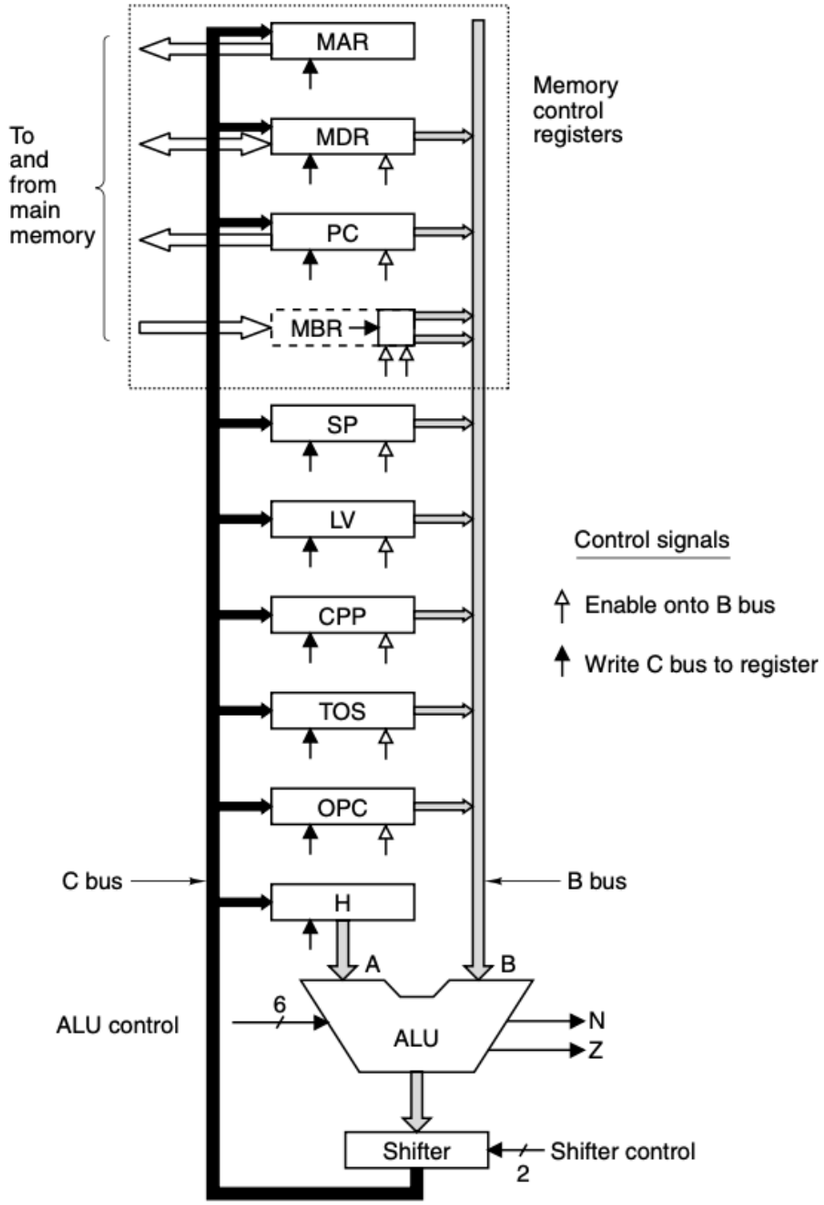


datapath.vhd

```
entity datapath is
  port (
    clk                 : in  std_logic;
    reset               : in  std_logic;
    alu_control         : in  alu_ctrl_type;
    c_to_reg_control    : in  c_ctrl_type;
    mem_control         : in  mem_ctrl_type;
    reg_to_b_control    : in  b_ctrl_type;
    mbr_reg_out         : out mbr_data_type;
    alu_n_flag          : out std_logic;
    alu_z_flag          : out std_logic;
    mem_data_we         : out std_logic;
    mem_data_in         : in  reg_data_type;
    mem_data_out        : out reg_data_type;
    mem_data_addr       : out reg_data_type;
    mem_instr_in        : in  mbr_data_type;
    mem_instr_addr      : out reg_data_type
    );
end entity datapath;

architecture behavioral of datapath is
  -- Come la implementiamo?
end architecture behavioral;
```

Componenti



Per implementare l'unità operativa dobbiamo scrivere in VHDL il codice relativo:

- ai registri;
- all'ALU;
- ai bus.

Registri

I registri sono implementati con un singolo `process`, che ne gestisce la logica di reset e quella di scrittura mediante il bus `C`:

datapath.vhd

```
-- Registers
signal sp_reg  : reg_data_type;
signal lv_reg  : reg_data_type;
signal cpp_reg : reg_data_type;
[...]
```

common_defs.vhd

```
--! Processor register data width
constant reg_data_width : positive :=
↪  32;
[...]
--! Processor register data
subtype reg_data_type is
  std_logic_vector(reg_data_width - 1
  ↪  downto 0);
[...]
--! C control MAR bit
constant c_ctrl_mar : natural := 0;
--! C control MDR bit
constant c_ctrl_mdr : natural := 1;
[...]
```

datapath.vhd

```
-- Processor registers
reg_proc : process(clk) is
begin
  if rising_edge(clk) then
    if reset = '1' then
      mar_reg <= (others => '0');
      pc_reg  <= (others => '0');
      [...]
    else
      if c_to_reg_control(c_ctrl_mar) = '1' then
        mar_reg <= c_bus;
      end if;
      if c_to_reg_control(c_ctrl_pc) = '1' then
        pc_reg <= c_bus;
      end if;
      [...]
    end if;
  end if;
end process reg_proc;
```

Convenzioni sui Registri

Alcuni registri sono cablati in modo da poter essere usati solo per uno scopo specifico:

- i registri dell'interfaccia con la memoria:
  - `MAR` - memory address register;
  - `MDR` - memory data register;
  - `PC` - program counter;
  - `MBR` - memory byte register;
- il registro che mantiene il primo operando dell'ALU:
  - `H` - holding.

Gli altri registri sono funzionalmente equivalenti, ed i loro nomi sono assegnati sulla base dell'uso che se ne fa nel microprogramma:

- `SP` - stack pointer;
- `LV` - local variables;
- `CPP` - constant pool pointer;
- `TOS` - top of stack;
- `OPC` - scratch register.

ALU

L'implementazione dell'ALU è puramente behavioral:

alu.vhd

```
entity alu is
  port (
    --! ALU control
    control        : in  alu_ctrl_type;
    --! ALU operand A
    operand_a      : in  reg_data_type;
    --! ALU operand B
    operand_b      : in  reg_data_type;
    --! ALU result
    sh_result      : out reg_data_type;
    --! Negative flag
    negative_flag : out std_logic;
    --! Zero flag
    zero_flag      : out std_logic
    );
end entity alu;

architecture behavioral of alu is
  [...]
end architecture behavioral;
```

F_0	F_1	ENA	ENB	INVA	INC	Function
0	1	1	0	0	0	A
0	1	0	1	0	0	B
0	1	1	0	1	0	$\bar{A}$
1	0	1	1	0	0	$\bar{B}$
1	1	1	1	0	0	A + B
1	1	1	1	0	1	A + B + 1
1	1	1	0	0	1	A + 1
1	1	0	1	0	1	B + 1
1	1	1	1	1	1	B – A
1	1	0	1	1	0	B – 1
1	1	1	0	1	1	–A
0	0	1	1	0	0	A AND B
0	1	1	1	0	0	A OR B
0	1	0	0	0	0	0
1	1	0	0	0	1	1
1	1	0	0	1	0	–1

Bus

A ciascun bus corrisponde un `signal` VHDL:

datapath.vhd

```
signal a_bus : reg_data_type;
signal b_bus : reg_data_type;
signal c_bus : reg_data_type;
```

Il multiplexer che disciplina l'accesso al bus `B` è implementato utilizzando il costrutto `with/select`:

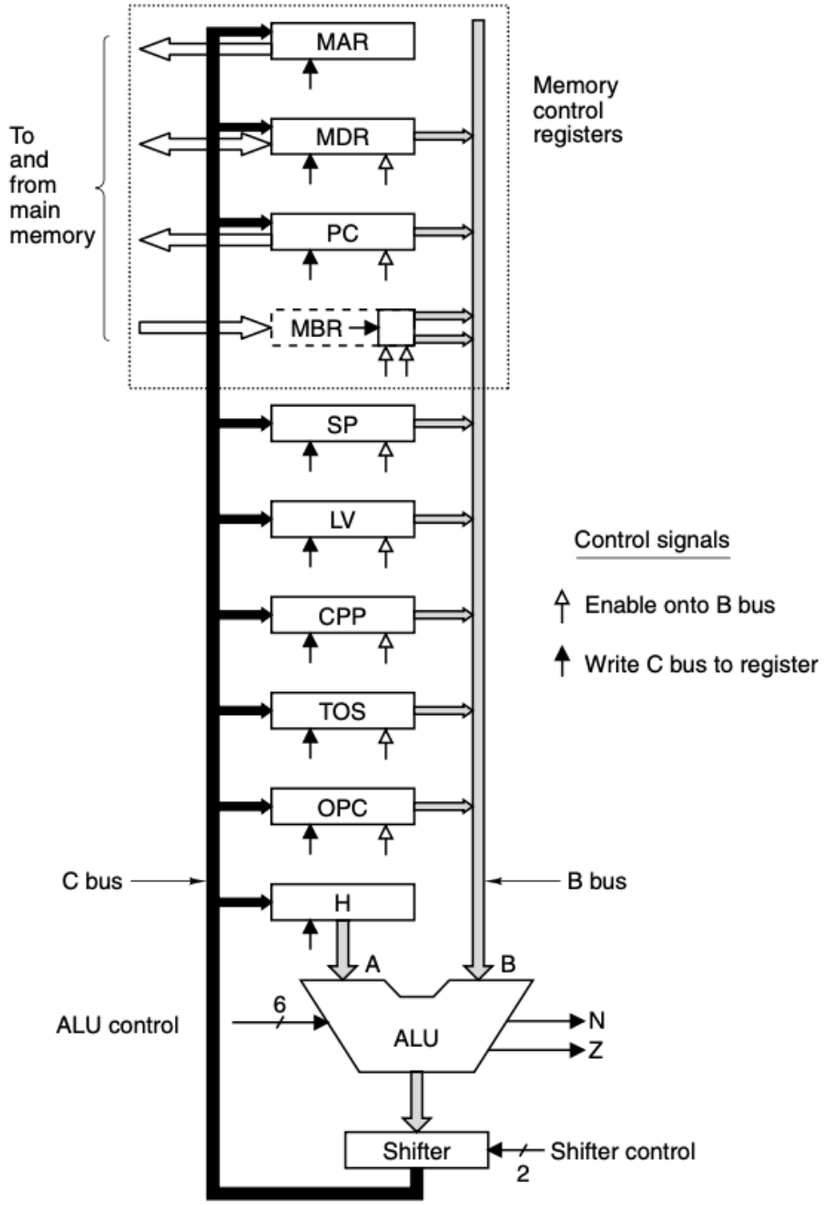
datapath.vhd

```
with reg_to_b_control select b_bus <=
  mdr_reg          when b_ctrl_mdr,
  pc_reg           when b_ctrl_pc,
  mbr_s            when b_ctrl_mbr,
  mbr_u            when b_ctrl_mbru,
  sp_reg           when b_ctrl_sp,
  lv_reg           when b_ctrl_lv,
  cpp_reg          when b_ctrl_cpp,
  tos_reg          when b_ctrl_tos,
  opc_reg          when b_ctrl_opc,
  (others => '0')  when others;
```

Logica tri-state

In altri corsi è presentato anche un altro modo di disciplinare l'accesso ai bus: facendo uso di buffer tri-state. Gli FPGA che utilizziamo non dispongono però di buffer tri-state se non ai pad di IO.

Ricapitolando

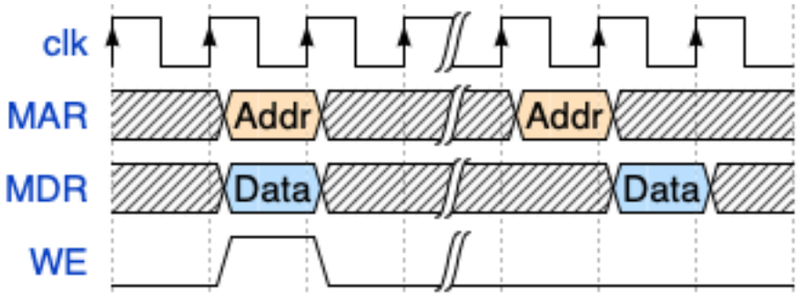


Abbiamo implementato:

- i registri con un `process`;
- l'ALU con un `component` con architettura behavioral;
- la disciplina di accesso al bus `B` con un costrutto `with/select`.

Queste parti evolvono **concorrentemente**.

Protocollo con la Memoria



Consideriamo l'interfaccia dati `MAR`/`MDR` (indirizzo e dato su 32 bit):

- Per eseguire una scrittura, l'indirizzo e il dato devono essere disponibili nei registri `MAR` e `MDR` al fronte di salita in cui è alto il segnale `WE`.
- Quando si opera in lettura (`WE` basso), il dato è disponibile nel registro `MDR` al fronte di clock successivo rispetto a quello in cui l'indirizzo è posto nel registro `MAR`.

Protocollo con la Memoria (2)

La lettura e la generazione del segnale di *write enable* sono implementate in un `process`:

datapath.vhd

```
-- Processor registers
reg_proc : process(clk) is
begin
  [...]
  if c_to_reg_control(c_ctrl_mdr) = '1' then
    mdr_reg <= c_bus;
  elsif rd_ff = '1' then
    mdr_reg <= mem_data_in;
  end if;
  [...]
  wr_ff <= mem_control(mem_ctrl_write);
end process;
```

I registri dell'interfaccia e il segnale di *write enable* sono assegnati alle uscite dell'`entity`:

datapath.vhd

```
-- Output
mem_data_out  <= mdr_reg;
mem_data_addr <= mar_reg;
mem_data_we   <= wr_ff;
```

Interfaccia istruzioni

L'interfaccia istruzioni `PC/MBR` (indirizzo su 32 bit, dato su 8 bit) opera in modo simile in lettura, e non prevede operazioni di scrittura.

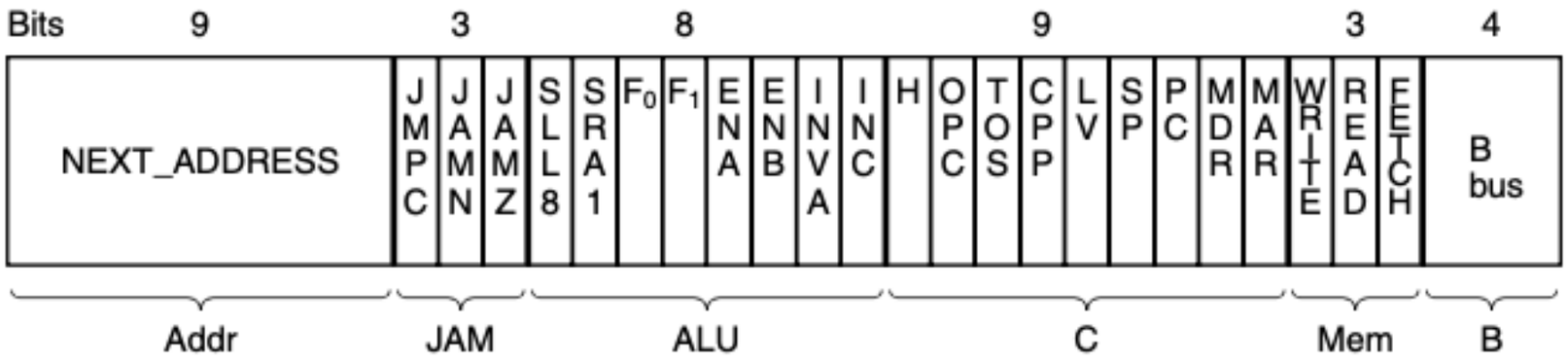
Segnali di Controllo

Per controllare l'unità operativa sono necessari:

- 9 segnali per controllare la scrittura dei dati dal bus `C` ai registri;
- 4 segnali per controllare quale registro è collegato al bus `B` e va in ingresso all'ALU;
- 8 segnali per controllare l'ALU;
- 2 segnali per abilitare lettura/scrittura sull'interfaccia `MAR`/`MDR`;
- 1 segnale per abilitare il fetch sull'interfaccia `PC`/`MBR`.

Inoltre ogni microistruzione deve contenere dei bit aggiuntivi per indicare la microistruzione successiva e per il controllo di flusso.

Formato delle Microistruzioni



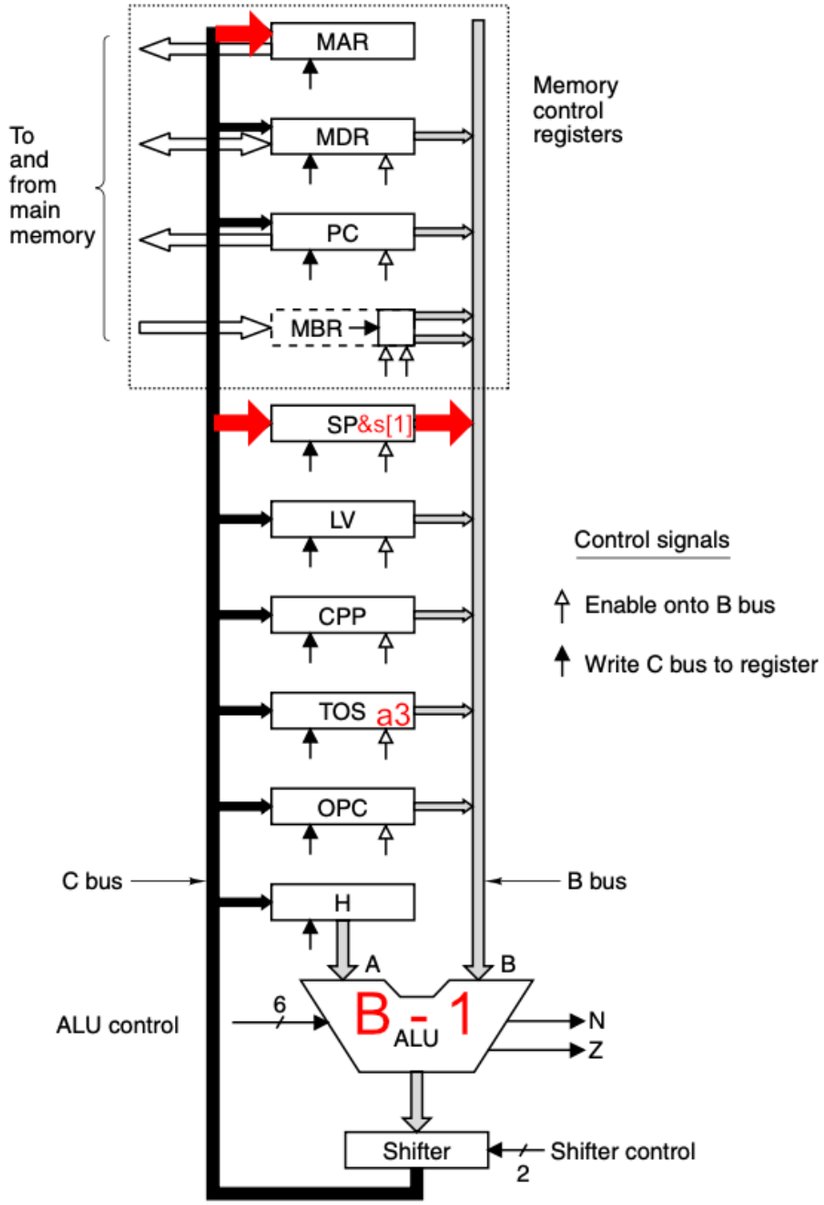
- `Addr` - Indirizzo di una potenziale prossima microistruzione;
- `JAM` - Determina come è selezionata la prossima microistruzione;
- `ALU` - Controllo dell'ALU e dello shifter;
- `C` - Controlla quali registri vengono scritti dal bus `C`;
- `Mem` - Controlla le operazioni di memoria;
- `B` - Seleziona il registro connesso al bus `B`.

Esempio: l'Istruzione IADD

Trascurando per ora i campi Addr e JAM, siamo in grado di codificare gli altri campi delle microistruzioni:

1.
   - ALU: 110110 (decremento dell'operando B);
   - C: 000001001 (bus C scrive su SP e MAR);
   - Mem: 010 (lettura dati);
   - B: 0100 (SP controlla il bus B).
2.
   - ALU: 010100 (l'operando B passa invariato);
   - C: 100000000 (bus C scrive su H);
   - Mem: 000 (nessuna operazione di memoria);
   - B: 0111 (TOS controlla il bus B).
3.
   - ALU: 111100 (somma degli operandi A e B);
   - C: 001000010 (bus C scrive su MDR e TOS);
   - Mem: 100 (scrittura dati);
   - B: 0000 (MDR controlla il bus B).

Esempio: l'Istruzione IADD - Microistruzione 1

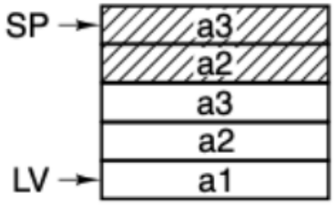


1

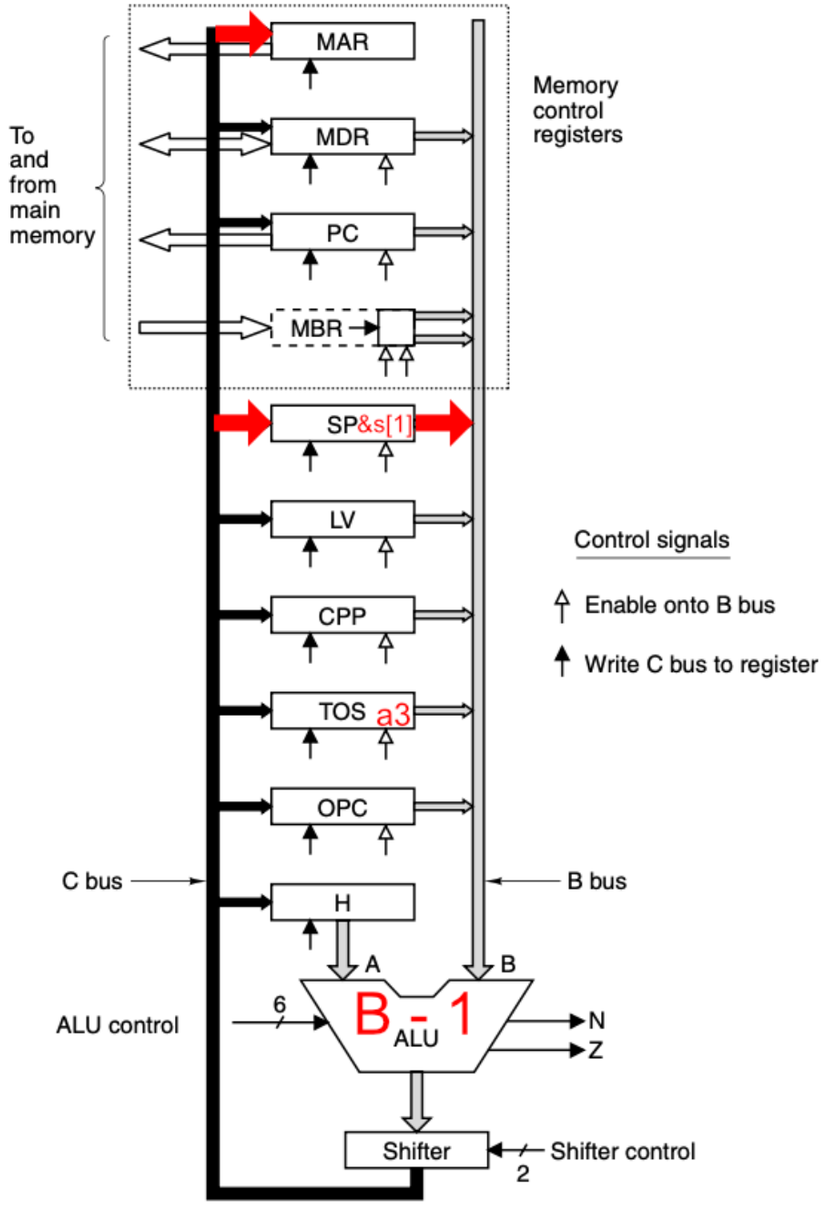
- ALU: 110110
- C: 000001001
- Mem: 010
- B: 0100

ajvm.mal

```
MAR = SP = SP - 1; rd
```

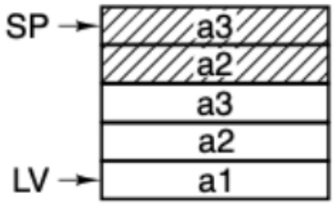


Esempio: l'Istruzione IADD - Microistruzione 1 (2)

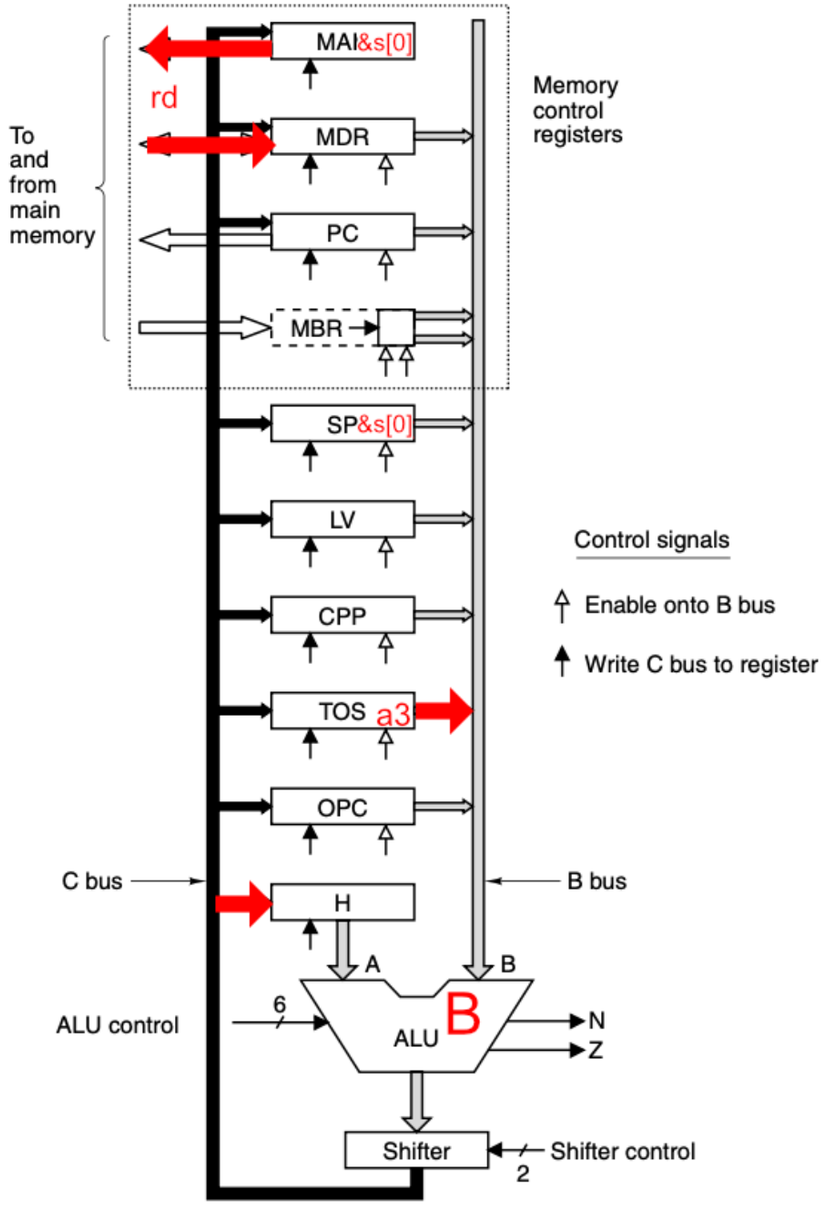


Notiamo due cose importanti:

1. la microistruzione ha predisposto il percorso nel datapath, ma i suoi effetti non sono visibili fino al fronte di clock successivo, quando i registri vengono scritti;
2. il segnale di read `rd` diventa alto al fronte di clock successivo, in quanto è solo allora che l'indirizzo a cui eseguire la lettura diventa disponibile sul registro `MAR`.



Esempio: l'Istruzione IADD - Microistruzione 2

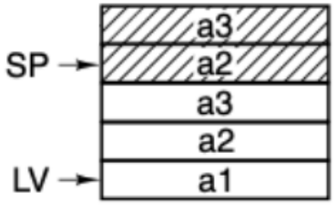


2

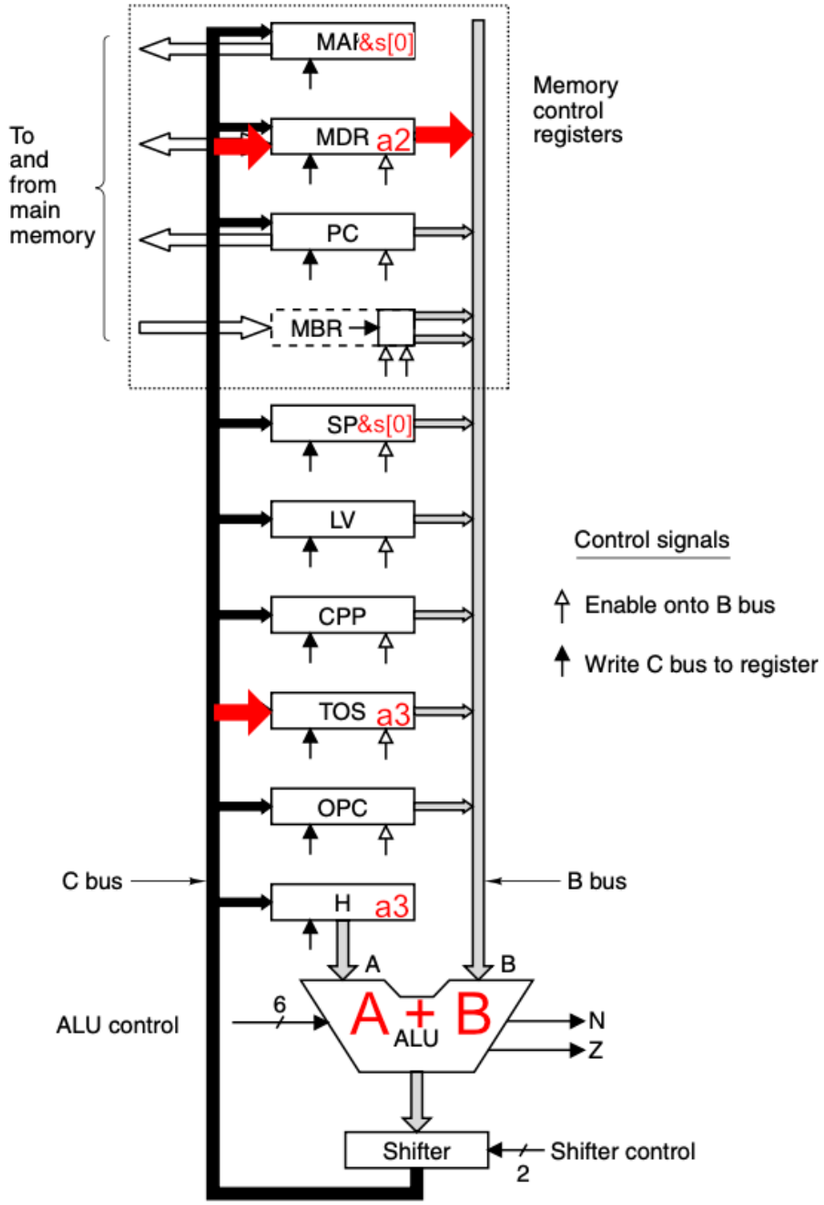
- ALU: 010100
- C: 100000000
- Mem: 000
- B: 0111

ajvm.mal

```
H = TOS
```



Esempio: l'Istruzione IADD - Microistruzione 3

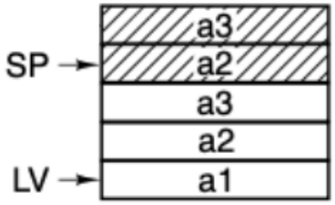


3

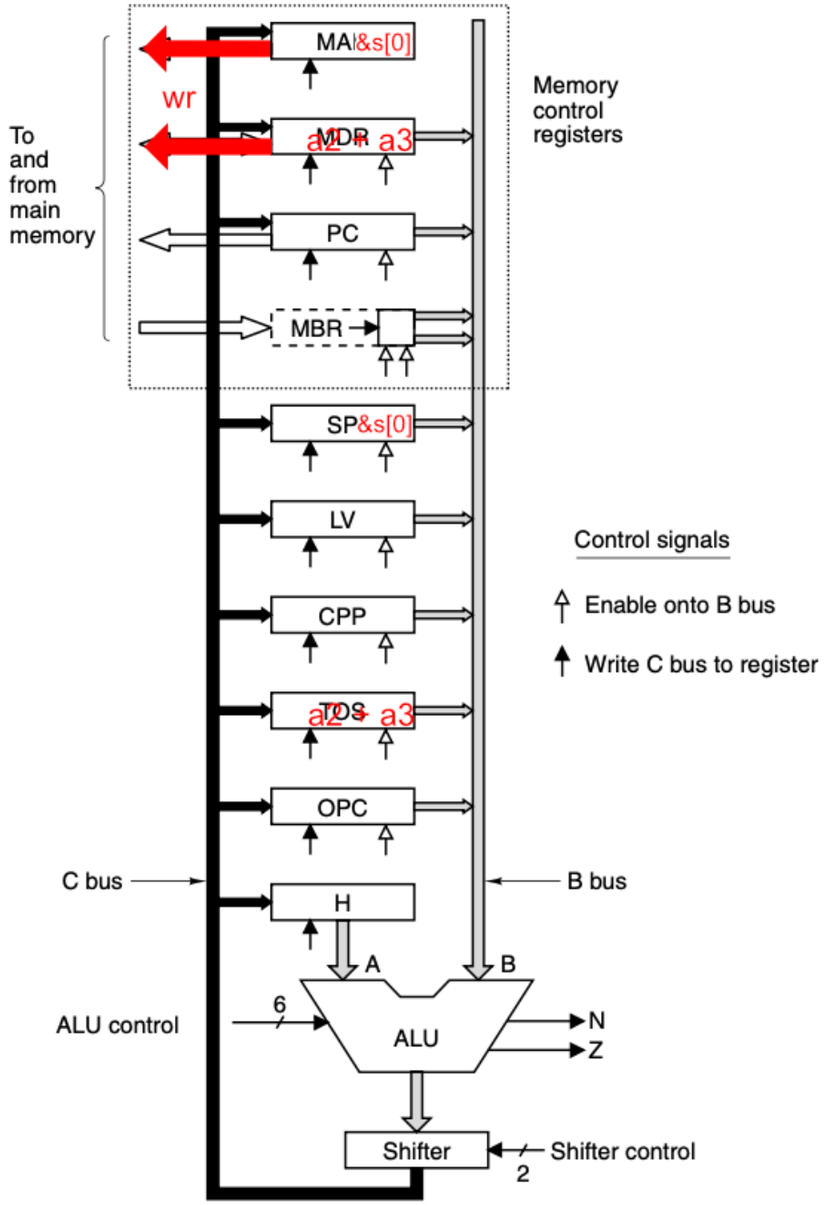
- ALU: 111100
- C: 001000010
- Mem: 100
- B: 0000

ajvm.mal

```
MDR = TOS = MDR + H; wr
```



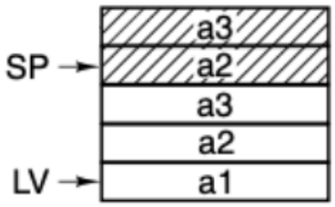
Esempio: l'Istruzione IADD - Ciclo 4



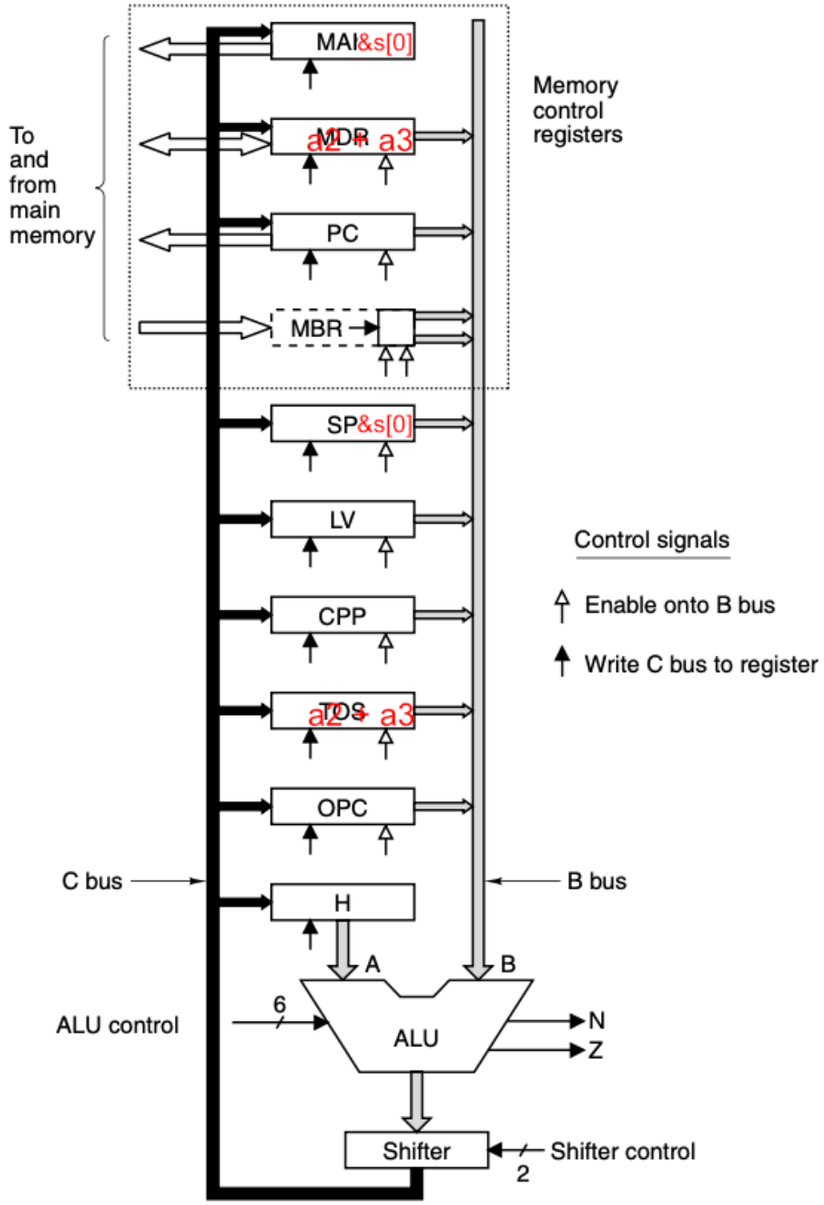
Le microistruzioni dell'IADD sono terminate. Come vedremo in seguito, dopo ciascuna istruzione viene eseguita una microistruzione speciale (main) che ha il compito di:

1. prelevare un byte dalla memoria istruzioni;
2. saltare alla prossima microistruzione (cioè la prima della sequenza relativa alla prossima istruzione).

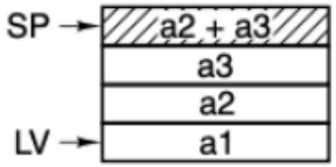
Sono ancora in corso gli effetti sulla memoria dell'istruzione IADD.



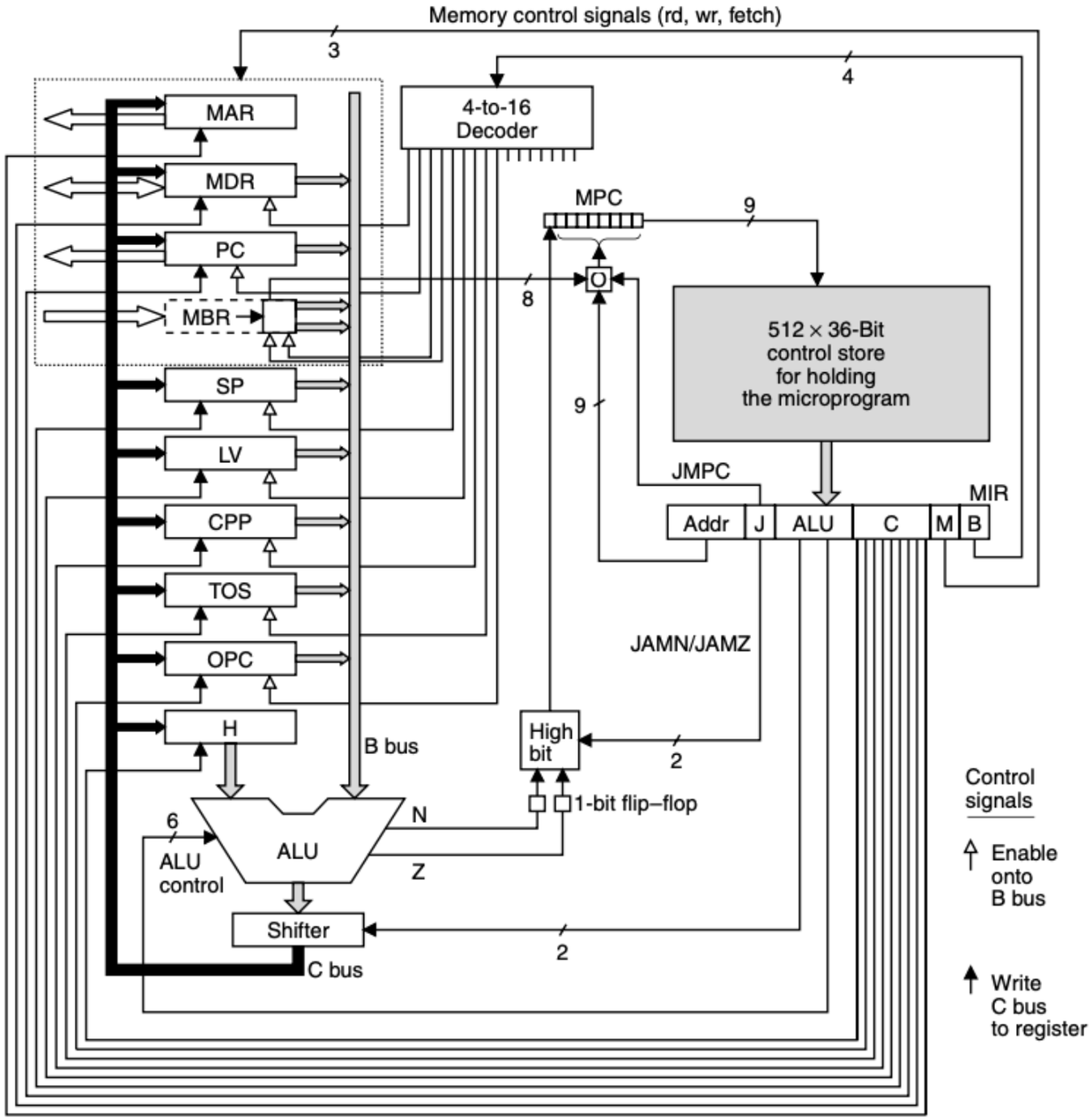
Esempio: l'Istruzione IADD - Ciclo 5



In questo ciclo si conclude la scrittura e viene eseguita la prima microistruzione dell'istruzione successiva; gli effetti sulla memoria dell'istruzione IADD diventano visibili.



Unità di Controllo



L'unità di controllo del Mic-1 si comporta come un *sequencer*, producendo in ciascun ciclo:

1. lo stato dei segnali di controllo;
2. l'indirizzo della prossima microistruzione da eseguire.

Control Store

Le microistruzioni sono memorizzate in una ROM interna all'unità di controllo, denominata *control store*:

control_store.vhd

```
entity control_store is
  port (
    --! Address of the desired word
    address : in  ctrl_str_addr_type;
    --! Content of the addressed word
    word    : out ctrl_str_word_type
    );
end entity control_store;
```

control_store.vhd

```
constant words : ctrl_str_type := (
--BEGIN_WORDS_ENTRY
0 => "00000011000000000000000000000001001",
1 => "01011110000000000000000000000001001",
2 => "00000000000000000000000000000000000",
[...]
--END_WORDS_ENTRY
);
```

common_defs.vhd

```
--! Control store address
subtype ctrl_str_addr_type is std_logic_vector(ctrl_str_addr_width - 1 downto 0);
--! Control store word
subtype ctrl_str_word_type is std_logic_vector(ctrl_str_word_width - 1 downto 0);
[...]
--! Control store word
subtype ctrl_str_word_type is std_logic_vector(ctrl_str_word_width - 1 downto 0);
[...]
--! Control store content
type ctrl_str_type is array (ctrl_str_words - 1 downto 0) of ctrl_str_word_type;
```

Altra Logica di Controllo

Inoltre, l'unità di controllo contiene:

- i registri `MIR` e `MPC` di interfaccia con il control store;
- il decoder per i segnali di controllo del bus `B`;
- la logica per determinare l'indirizzo della prossima microistruzione.

A livello microprogramma, i salti sono controllati mediante il campo `JAM`: i due bit `JAMN` e `JAMZ` sono combinati con i valori dei flag `N` e `Z` dell'ALU, e permettono di modificare il bit più significativo dell'`MPC`:

```
MPC[8] = (JAMZ AND Z) OR (JAMN AND N) OR Addr[8]
```

Se il bit `JMPC` è asserito, gli 8 bit meno significativi di `MPC` sono messi in `OR` bit-a-bit con gli 8 bit contenuti nel registro `MBR`:

```
MPC[7:0] = Addr[7:0] OR (MBR[7:0] AND JMPC)
```

Altra Logica di Controllo (2)

Il decoder è implementato con un `process` puramente combinatorio:

control_unit.vhd

```
-- B_BUS control decoder
reg_to_b_decoder : process(mir_reg(ctrl_b'range)) is
begin
  reg_to_b_decoder_out <= (others => '0');

  if unsigned(mir_reg(ctrl_b'range)) < b_ctrl_width then
    reg_to_b_decoder_out(to_integer(unsigned(mir_reg(ctrl_b'range)))) <= '1';
  end if;
end process reg_to_b_decoder;
```

La logica di salto è implementata con delle assegnazioni su un *virtual register*, i.e. dei semplici `signal` che rimangono stabili fino al fronte di clock successivo:

control_unit.vhd

```
-- MPC virtual register
ctrl_nxt_addr_no_msb <= mir_reg(ctrl_nxt_addr_no_msb_type'range);
jmpc_addr <= ctrl_nxt_addr_no_msb or mbr_reg_in
             when mir_reg(ctrl_jmpc) = '1'
             else ctrl_nxt_addr_no_msb;
high_bit <= (alu_n_flag and mir_reg(ctrl_jamn))
            or (alu_z_flag and mir_reg(ctrl_jamz));
mpc_virtual_reg <= (mir_reg(ctrl_nxt_addr_msb) or high_bit) & jmpc_addr;
```

Sequencing delle Microistruzioni

L'unità di controllo si comporta effettivamente come un *sequencer*.

ajvm.mal

```
main:
    PC = PC + 1; fetch; goto (MBR)
```

ajvm.mal

```
iadd = 0x65:
    MAR = SP = SP - 1; rd
    H = TOS
    MDR = TOS = MDR + H; wr; goto main
```

1. Dopo una fase di inizializzazione, viene eseguita la microistruzione `main` che incrementa il `PC`, esegue il fetch di un'istruzione e imposta come prossima microistruzione quella "puntata" dal registro `MBR`.
2. Se ad esempio l'istruzione prelevata su `MBR` è `0x65`, viene eseguita la microistruzione all'indirizzo `0x65` del control store, la `IADD`.
3. Dopo le tre microistruzioni della `IADD` viene rieseguita la microistruzione alla label `main`, che esegue il fetch della prossima istruzione.

Esempi di Istruzioni IJVM

Esempi di codici di movimento:

ajvm.mal

```
iload = 0x15:
        H = LV
        MAR = MBRU + H; rd
iload_cont:
        MAR = SP = SP + 1
        PC = PC + 1; fetch; wr
        TOS = MDR; goto main

istore = 0x36:
        H = LV
        MAR = MBRU + H
istore_cont:
        MDR = TOS; wr
        SP = MAR = SP - 1; rd
        PC = PC + 1; fetch
        TOS = MDR; goto main
```

Esempi di codici di salto:

ajvm.mal

```
ijvm_goto = 0xA7:
        OPC = PC - 1
goto_cont:
        PC = PC + 1; fetch
        H = MBR << 8
        H = MBRU OR H
        PC = OPC + H; fetch
        goto main

[...]

ifeq = 0x99:
        MAR = SP = SP - 1; rd
        OPC = TOS
        TOS = MDR
        Z = OPC; if (Z) goto T; else goto F

[...]

T:
        OPC = PC - 1; fetch; goto goto_cont

F:
        PC = PC + 1
        PC = PC + 1; fetch
        goto main
```

Esempi di Istruzioni IJVM (2)

Consideriamo l'istruzione `ILOAD varnum`, con `varnum` variabile da caricare su stack (offset rispetto al puntatore in `LV`), codificata su 1 byte:

1. il puntatore contenuto in `LV` è scritto in `H`;
2. in `MAR` è posta la somma base `LV` + offset `MBRU`, e si abilita la lettura;
   - `MBRU` significa: contenuto di `MBR`, come unsigned;
   - il contenuto di `MBR` è il risultato dell'ultimo `fetch` eseguito dalla microistruzione `main`;
3. `MAR` e `SP` sono posti ad una nuova posizione vuota su stack (dove sarà fatto il push del nuovo elemento); inoltre il dato di cui eseguire il push è ora disponibile in `MDR`;
4. `PC` è incrementato e si fa il `fetch` della prossima istruzione (in modo che `main` possa eseguire il `goto` alla microistruzione corretta); si esegue anche la scrittura (push) in memoria;
5. il contenuto di `TOS` è aggiornato con il dato di cui si è eseguito il push.

Installazione dei Tool

In caso di dubbi, fare riferimento alle dispense per la procedura dettagliata.

```
$ sudo apt install gcc g++ make cmake git ghdl-gcc gtkwave
$ sudo apt update
$ sudo apt install openjdk-11-jdk
$ cd ~
$ mkdir tools
$ mkdir esercitazione_mic
$ cd esercitazione_mic
$ git clone https://github.com/albmoriconi/mal.git
$ cd mal
$ ./gradlew build
$ cd ..
$ git clone https://github.com/albmoriconi/ajvm.git
$ cd ajvm
$ ./gradlew build
$ cd ..
$ tar -xvf mal/build/distributions/mal.tar -C ~/tools/
$ tar -xvf ajvm/build/distributions/ajvm.tar -C ~/tools/
$ echo 'export PATH="$HOME/tools/ajvm/bin:$HOME/tools/mal/bin:$PATH"' >> ~/.bashrc
$ git clone https://github.com/albmoriconi/amic-0.git
$ cd amic-0
$ cmake -B build
```

Struttura del Progetto

La directory principale del repository amic-0 è organizzata come segue:

- `src` - codice sorgente
  - `main` - implementazione (subdirectory ordinate per linguaggio)
  - `test` - testbench (subdirectory ordinate per linguaggio)
- `util` - utility necessarie per il flow di sviluppo (subdirectory ordinate per linguaggio)
- `build` - contiene i file prodotti dal build system (questa directory deve essere creata manualmente)

Ogni directory che contiene codice (eventualmente nelle subdirectory) contiene anche un file `CMakeLists.txt` che serve al build system per tenere traccia dei sorgenti e aggiungerli ai target opportuni.

Flow di Sviluppo

1. Per modificare microprogramma e programma bisogna modificarne i file e lanciare i target make relativi.
   - `src/main/mal/ajvm.mal` - microprogramma
     - `cmake --build build --target create_control_store`
   - `src/main/ajvm/program.ajvm` - programma
     - `cmake --build build --target create_ram`
2. Per lanciare la suite di test va lanciato il target `check`
3. Le forme d'onda (visualizzabili con `gtkwave`) sono salvate nella directory `build/src/test/vhdl`.

Per aggiungere altri testbench alla suite:

1. salvarli nella directory `test/vhdl`;
2. modificare il file `test/vhdl/CmakeLists.txt` aggiungendo alla macro `add_vhdl_test_sources` un'entry relativa al nuovo testbench.

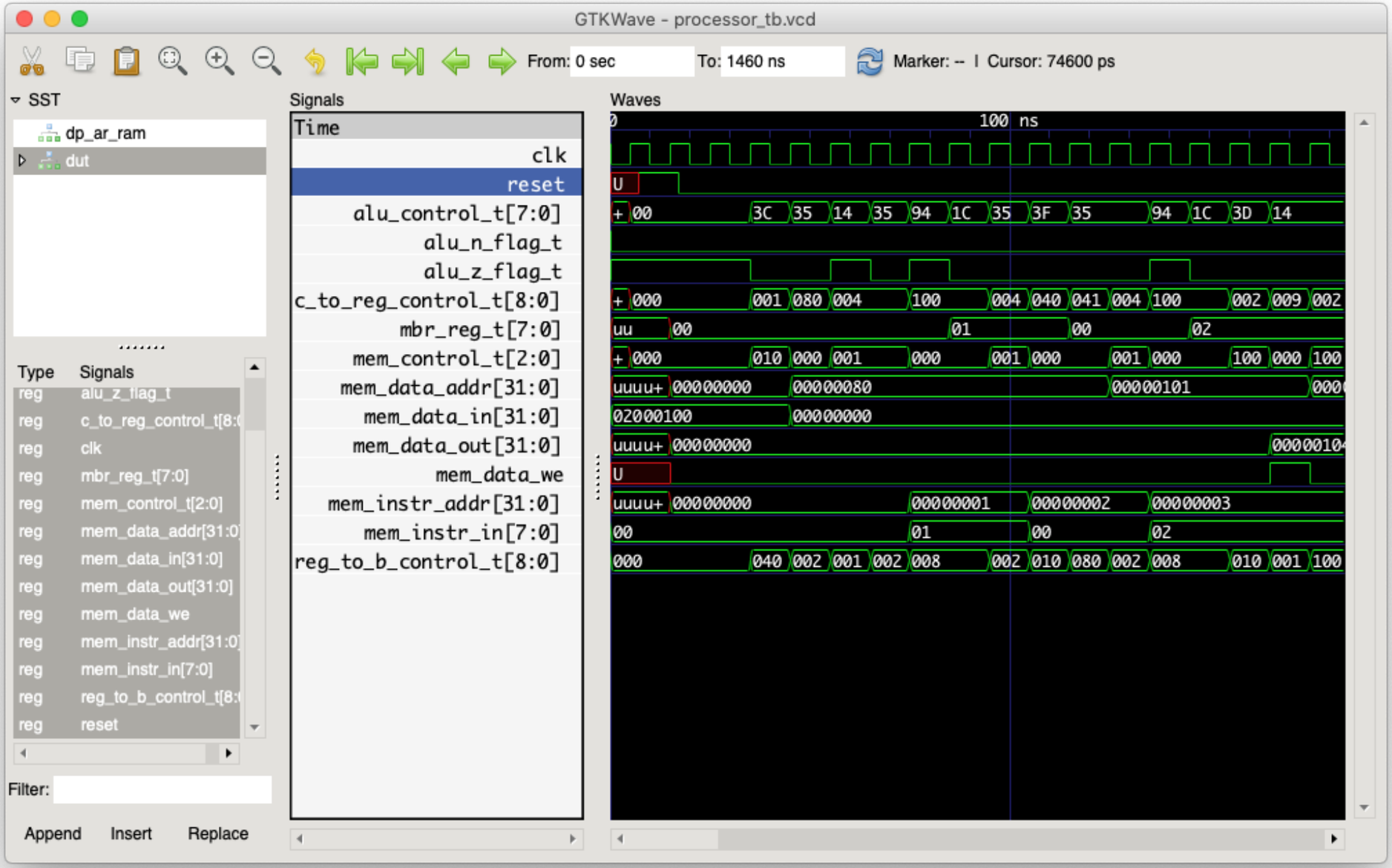
Risultati dei Test (Esempio)

```
make — tmux — 100×30
Built target src.test.vhdl.processor_tb
Scanning dependencies of target src.test.vhdl.alu_tb
analyze /Users/albmoriconi/Progetti/amic-0/src/test/vhdl/alu_tb.vhd
elaborate alu_tb
Built target src.test.vhdl.alu_tb
Scanning dependencies of target src.test.vhdl.control_unit_tb
analyze /Users/albmoriconi/Progetti/amic-0/src/test/vhdl/control_unit_tb.vhd
elaborate control_unit_tb
Built target src.test.vhdl.control_unit_tb
Scanning dependencies of target src.test.vhdl.datapath_tb
analyze /Users/albmoriconi/Progetti/amic-0/src/test/vhdl/datapath_tb.vhd
elaborate datapath_tb
Built target src.test.vhdl.datapath_tb
Scanning dependencies of target check
Test project /Users/albmoriconi/Progetti/amic-0/build
    Start 1: src.test.vhdl.alu_tb
1/4 Test #1: src.test.vhdl.alu_tb .............   Passed    0.37 sec
    Start 2: src.test.vhdl.control_unit_tb
2/4 Test #2: src.test.vhdl.control_unit_tb ....   Passed    0.19 sec
    Start 3: src.test.vhdl.datapath_tb
3/4 Test #3: src.test.vhdl.datapath_tb ........   Passed    0.18 sec
    Start 4: src.test.vhdl.processor_tb
4/4 Test #4: src.test.vhdl.processor_tb .......   Passed    0.21 sec

100% tests passed, 0 tests failed out of 4

Total Test time (real) =   0.95 sec
Built target check
[~/P/a/build]                                                  (master|✔)
[0] 1:make* 2:fish-                                        19:13 25-Nov-19
```

Waveform dei Testbench (Esempio)



Tracce degli Esercizi

1. Simulare l'esecuzione di un programma contenente due istruzioni IJVM.
   - È necessario modificare il programma (`program.ajvm`) e ricreare la RAM.
   - Dunque, si può aggiungere un nuovo testbench alla suite o modificare quello già esistente (`processor_tb.vhd`).
2. Modificare l'operazione eseguita da un'istruzione IJVM.
   - È necessario modificare il microprogramma (`ajvm.mal`) e ricreare il control store.
   - Che succede modificando la `IADD`? Come possiamo farle eseguire una sottrazione?

Esercizi **Facoltativi**

3. Sintetizzare su FPGA il processore.
   - Così come implementato, potrebbe essere "troppo grande" per alcune board;
   - in questo caso, conviene ridurre la dimensione della RAM:
     - nel file `dp_ar_ram.vhd` va ridotta la distanza tra l'area codice e il constant pool;
     - nel file `datapath.vhd` va corretto il puntatore al constant pool e quello allo stack;
     - infine, nel file `common_defs.vhd` va ridotta la dimensione della RAM, impostando `dp_ar_ram_size` e.g. a 64 word da 32 bit.
4. Collegare al processore una o più periferiche di IO.
   - Un esempio completo è fornito nel branch `pwm` del repository git;
   - all'instruction set sono state aggiunte le istruzioni `gload` e `gstore` per interfacciarsi con i registri memory mapped delle periferiche;
   - il file `device_mux.vhd` implementa una logica elementare per assegnare parte dello spazio di indirizzamento alle periferiche di IO.